TenderHub Future Configurations Manual

Secondary Server Automated Failover & Render Paid Plan Upgrade Roadmap

FUTURE CONFIGURATION

Engineering Directive: Automated switching to the secondary MongoDB cluster is **intentionally disabled** under current deployment parameters. The platform operates strictly in single-cluster mode with the primary database. Automatic failover will be activated upon upgrading to a Render paid service plan.

1. Executive Summary & Architecture Context

TenderHub is engineered with an enterprise-grade disaster recovery and hot-standby architecture. While the platform contains complete code and socket redirection logic for automated database failover, this functionality has been deferred until the hosting infrastructure on **Render** is upgraded to a dedicated paid plan.

2. Render Free Tier vs. Paid Plan Technical Matrix

Architecture Dimension	Render Free Tier (Current)	Render Paid Plan (Future Required)	Impact on Automated Failover
Process Lifecycle	Spins down after 15 minutes of inactivity	Always-On persistent daemon (No idle sleep)	Free tier wake-up latency mimics database timeouts, causing false failovers.
Outbound IP Routing	Dynamic shared IP pool across Render users	Static Outbound IP / Dedicated Egress	MongoDB Atlas Network Access requires persistent IP whitelisting for both clusters.
Connection Pooling	Limited socket descriptors per dyno	High-capacity connection pools	Simultaneous hot socket connections to primary and secondary require dedicated resources.
Background Workers	Shared CPU threads, throttled I/O	Dedicated BullMQ worker dynos	Scheduled cron purges and synchronization jobs run with guaranteed SLA.
Zero-Downtime Deploys	Cold restarts with brief request drop	Health-checked Blue/Green rolling deploys	Permits seamless socket migration between primary and secondary database clusters.

3. Rationale for Pausing Automated Switching on Free Tier

- **Prevention of False Failovers:** On the free tier, when an application instance sleeps and receives a web request, cold-start initialization can take 20–50 seconds. A 6-second timeout threshold would trigger unnecessary failovers to the backup server during standard cold starts.
- **Split-Brain Avoidance:** Without a paid plan offering dedicated background workers, two separate instances could simultaneously write to different database clusters, leading to state desynchronization.
- **Strict Resource Allocation:** Free tier memory limits (512 MB) are optimized for serving active user traffic rather than maintaining two concurrent high-availability database driver pools.

4. Current Operational Baseline (Safe Single-Cluster Mode)

Current Active Configuration in `backend/src/config/db.js`:

- **Primary Cluster:** Exclusively connects to `MONGO_URI`.
- **Automated Switching:** Disabled. If primary cluster is unreachable, server logs the error and gracefully halts without switching sockets.
- **Disaster Recovery:** Fully operational on demand via manual runbooks (`npm run restore:db` , `npm run backup:full`).
- **Secondary Cloudflare R2:** Actively receiving async document mirrors and zero-delay expired tender purges.

TenderHub Future Configurations Manual

Implementation Blueprint & Step-by-Step Enablement Runbook

ACTIVATION RUNBOOK

5. The Automated Switching Logic (Technical Blueprint)

When the Render paid plan is acquired, the automated switching logic inside `backend/src/config/db.js` will be activated using the following battle-tested architecture:

```
// backend/src/config/db.js - Automated Failover Blueprint
try {
  logger.info('[Database] Connecting to Primary MongoDB cluster...');
  activeMongoUri = primaryUri;
  isUsingSecondaryDb = false;
  await mongoose.connect(primaryUri, { serverSelectionTimeoutMS: 6000 });
} catch (primaryError) {
  logger.error(`❌ Primary MongoDB Connection Failed: ${primaryError.message}`);
  if (secondaryUri && secondaryUri.trim()) {
    logger.warn(`⚠️ [FAILOVER ACTIVATED] Switching to Secondary MongoDB Cluster...`);
    activeMongoUri = secondaryUri;
    isUsingSecondaryDb = true;
    await mongoose.connect(secondaryUri, { serverSelectionTimeoutMS: 8000 });
    logger.info(`🎉 [FAILOVER SUCCESS] Online using Secondary MongoDB Cluster!`);
    return;
  }
  process.exit(1);
}
```

6. Render Paid Plan Upgrade Requirements

Before activating automated switching, ensure the following Render configurations are completed:

1. **Upgrade Web Service to Starter ($7/mo) or Standard ($25/mo):**
   - Disables auto-sleep completely (ensures 100% uptime with 0s wake-up latency).
   - Provides 1 GB to 2 GB dedicated RAM and 0.5 to 1 dedicated vCPU.
2. **Configure Secondary MongoDB Atlas Whitelisting:**
   - Add Render outbound IP addresses to the Network Access whitelist of the secondary MongoDB cluster.
3. **Set Environment Variables in Render Dashboard:**

```
# Render Dashboard -> Environment Variables
MONGO_URI=[REDACTED]
SECONDARY_MONGO_URI=[REDACTED]
ENABLE_AUTOMATED_DB_FAILOVER=true
```

7. Step-by-Step Activation Runbook

Step #	Action	Command / Procedure	Expected Outcome
Step 1	Purchase Render Paid Plan	Render Dashboard → Service Settings → Upgrade Plan (Starter/Standard)	Application is always-on; idle spin-down is permanently disabled.
Step 2	Provision Secondary Database	MongoDB Atlas → Deploy secondary M0/M10 cluster	Obtain `SECONDARY_MONGO_URI` connection string.
Step 3	Sync Baseline Data	`MONGO_URI=[REDACTED] npm run restore:db`	Secondary cluster populated with latest snapshot from secondary R2.
Step 4	Notify Engineering & Enable	Instruct agent to re-enable automated failover in `db.js`	6-second timeout threshold activated with automatic socket switching.

Step #	Action	Command / Procedure	Expected Outcome
Step 5	Execute Controlled Failover Drill	Temporarily pause primary cluster in Atlas dashboard	Server automatically logs `[FAILOVER ACTIVATED]` and stays online.

Authoritative Sign-Off: This future configuration roadmap has been officially cataloged in the TenderHub architecture repository. All technical prerequisites and code changes are isolated, documented, and ready for immediate deployment upon instruction.